Documentation de Conception

GL03



CAMATCHY Alexander
CANAGUY Vincent
GRAHAM Martha
GUINET Aster
LAISNÉ Matthew
MATER Hugo
NGONGANG Gaston
THOMAZIC Valentin
WENDELS Jérémy

Table des matières

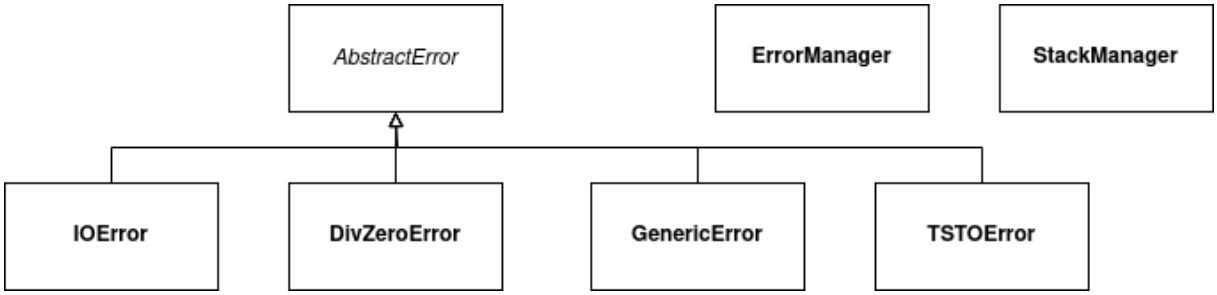
1. Architecture ____ -3-
   1.1. Classes du dossier Codegen ____ -3-
      1.1.1. Classes d'erreurs ____ -3-
      1.1.2. Classes de Labels ____ -3-
   1.2. Classes du dossier context ____ -4-
   1.3. Classes du dossier Syntax ____ -4-
   1.4. Classes du dossier Tools ____ -4-
   1.5. Classes du dossier Tree ____ -4-
      1.5.1. DeclField ____ -4-
      1.5.2. DeclMethod ____ -4-
      1.5.3. DeclParam ____ -4-
      1.5.4. DeclObject ____ -4-
      1.5.5. New Expr ____ -5-
         1.5.5.1. Cast ____ -5-
         1.5.5.2. InstanceOf ____ -5-
         1.5.5.3. MethodCall ____ -5-
         1.5.5.4. New ____ -5-
         1.5.5.5. NullLiteral ____ -5-
         1.5.5.6. ThisLiteral ____ -5-
      1.5.6. MethodAsmBody ____ -5-
      1.5.7. MethodBody ____ -5-
      1.5.8. Return ____ -6-
      1.5.9. Selection ____ -6-
2. Spécifications rajoutées ____ -6-
   2.1. Extension ____ -6-
3. Algorithmes et structures de données rajoutés ____ -6-

1. Architecture

1.1. Classes du dossier Codegen

1.1.1. Classes d'erreurs

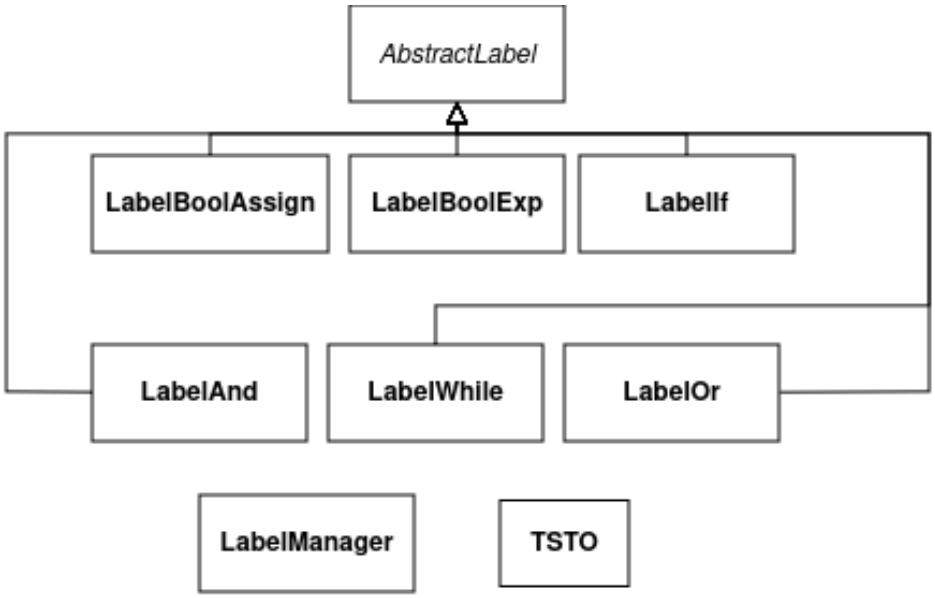
Cette ensemble de classes permet de gérer les erreurs et de générer les instructions assembleurs permettant de détecter ces erreurs. Chaque classes héritant de AbstractError ce sert du labelManager instancié dans le compiler afin de gérer la génération de label et des instructions correspondant aux erreurs.



1.1.2. Classes de Labels

Cette ensemble de classes permet de gérer les labels afin de conserver l'unicité de la manière la plus simple possible.

Chaque classe héritant de AbstractLabel possède un id et une méthode toString(). Le labelManager quant à lui contient des int représentant le nombre de label déjà existant de chaque type. Il s'en sert afin de créer les labels et de préfixé le nom de chacun d'entre eux.



1.2. Classes du dossier context

Aucune autre classe que celles déjà présentes n'ont été implémenté. Cependant quelque méthodes ont été rajouté afin de faciliter l'utilisation comme par exemple:

- Des `asType()` supplémentaire qui permettent de cast ou renvoyer dynamiquement une erreur.
- Une modification du `get()` de `EnvironnementExp` afin de rechercher les symboles également dans le parent
- Une méthode `getAllMethods()` dans `EnvironnementExp` pour récupérer tout les symboles parent y compris (récursivement) en écrasant ceux de même nom par les plus jeunes enfants.
- Une méthode `sameSignature()` facilitant la comparaison de signature de paramètre d'une méthode.

1.3. Classes du dossier Syntax

Aucune autre classe que celles déjà présentes n'ont été implémenté

1.4. Classes du dossier Tools

Aucune autre classe que celles déjà présentes n'ont été implémenté

1.5. Classes du dossier Tree

1.5.1. DeclField

Cette nouvelle classe hérite `AbstractDeclField`. Elle permet de gérer tout ce qui concerne la déclaration de champs (méthode classique: codeGen, verify, etc)

1.5.2. DeclMethod

Cette nouvelle classe hérite `AbstractDeclMethod`. Elle permet de gérer tout ce qui concerne la déclaration de méthodes (méthode classique: codeGen, verify, etc)

1.5.3. DeclParam

Cette nouvelle classe hérite `AbstractDeclParam`. Elle permet de gérer tout ce qui concerne la déclaration de paramètres dans les méthodes (méthode classique: codeGen, verify, etc).

1.5.4. DeclObject

Cette nouvelle classe hérite `AbstractDeclClass`. Elle permet de gérer tout ce qui concerne la déclaration de la classe `Object` dont hérite toute les class de Déca. Déclaration des méthodes et champs compris.

1.5.5. New Expr

Toutes les nouvelles classes ci-dessous hérite `AbstractExpr`

1.5.5.1. Cast

Elle permet de gérer tout ce qui concerne le cast en déca. (méthode classique: codeGen, verify, etc)

1.5.5.2. InstanceOf

Elle permet de gérer tout ce qui concerne le instanceof en déca. (méthode classique: codeGen, verify, etc)

1.5.5.3. MethodCall

Elle permet de gérer tout ce qui concerne l'appel de méthode en déca. (méthode classique: codeGen, verify, etc)

1.5.5.4. New

Elle permet de gérer tout ce qui concerne le new en déca. (méthode classique: codeGen, verify, etc)

1.5.5.5. NullLiteral

Elle permet de gérer tout ce qui concerne le null en déca. (méthode classique: codeGen, verify, etc)

1.5.5.6. ThisLiteral

Elle permet de gérer tout ce qui concerne le this en déca. (méthode classique: codeGen, verify, etc)

1.5.6. MethodAsmBody

Cette nouvelle classe hérite `AbstractMethodBody`. Elle permet de gérer tout ce qui concerne la méthode asm en déca. (méthode classique: codeGen, verify, etc)

1.5.7. MethodBody

Cette nouvelle classe hérite `AbstractMethodBody`. Elle permet de gérer tout ce qui concerne le body des méthodes déca. (méthode classique: codeGen, verify, etc)

1.5.8. Return

Cette nouvelle classe hérite `AbstractInst`. Elle permet de gérer tout ce qui concerne l'instruction return en déca. (méthode classique: codeGen, verify, etc)

1.5.9. Selection

Cette nouvelle classe hérite `AbstractLValue`. Elle permet de gérer tout ce qui concerne la sélection d'attributs en déca. (méthode classique: codeGen, verify, etc)

2. Spécifications rajoutées

2.1. Extension

L'extension « Snake » a été rajouté au projet en tant que sous module. Elle permet de lancé un snake codé en Java. Elle doit aussi permettre de générer le code assembleur du code Déca du Snake. Pour cela, elle utilise seulement des

3. Algorithmes et structures de données rajoutés

Pour l'instant aucun algorithme ou structure n'est implémenté en plus de ceux déjà spécifiés.